



smelt-ml: A Pure-Rust Machine Learning Framework Combining Spatial Modeling, Conformal Prediction, and Cache-Optimized Gradient Boosting

Francisco Parra

Universidad de Santiago de Chile

Abstract

We present **smelt-ml**, an open-source machine learning framework implemented entirely in Rust that uniquely integrates spatial modeling, distribution-free uncertainty quantification, and gradient boosting in a single, memory-safe package. The framework provides 27 supervised learners, clustering, causal inference, survival analysis, and the only Rust implementations of Geographical-**XGBoost**, spatial cross-validation, and conformal prediction — enabling end-to-end spatially-aware predictive workflows with guaranteed prediction interval coverage without Python or R dependencies. The gradient boosting implementations employ cache-optimized column-major histogram accumulation with **u8** packing. On single-threaded benchmarks (n from 500 to 100,000, 10 runs, mean $\pm$ std), **smelt-ml** is 1.4–8.3 $\times$ faster than official C++ libraries at research-typical scales ($n \leq 10,000$) and consumes an order of magnitude less memory, while C++ is faster at $n \geq 50,000$. The composable, trait-based architecture is inspired by **mlr3** (Lang, Binder, Richter, Schratz, Pfisterer, Coors, Au, Casalicchio, Kotthoff, and Bischl 2019) and **scikit-learn** (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, and Duchesnay 2011), with zero **unsafe** blocks across 13,000+ lines of code.

Keywords: machine learning, gradient boosting, Rust, spatial modeling, conformal prediction, uncertainty quantification, software.

1. Introduction

Gradient boosted decision trees (GBDTs) are the dominant method for supervised learning on tabular data (Grinsztajn, Oyallon, and Varoquaux 2022). The three most widely used implementations — **XGBoost** (Chen and Guestrin 2016), **LightGBM** (Ke, Meng, Finley, Wang,

Chen, Ma, Ye, and Liu 2017), and **CatBoost** (Prokhorenkova, Gusev, Vorobev, Dorogush, and Gulin 2018) — are implemented in C++ and have been optimized over many years. These libraries serve as the de facto baselines in the majority of applied machine learning studies.

Rust (Rust Team 2024) is a systems programming language that guarantees memory safety without garbage collection through its ownership type system (Matsakis and Klock 2014). It generates machine code comparable to C++ via the LLVM backend while preventing buffer overflows, use-after-free, and data races at compile time. Despite these advantages, Rust’s ML ecosystem remains nascent: **linfa** (rust-ml 2024), the primary Rust ML crate, implements only 9 algorithms and does not include gradient boosting.

Beyond the language gap, a practical gap exists in the spatial ML workflow: practitioners who need spatially-aware prediction with uncertainty quantification must currently combine multiple Python or R packages with incompatible APIs (e.g., **scikit-learn** for modeling, **geoxgboost** for spatial regression, **MAPIE** for conformal intervals, and custom spatial resampling code). No existing framework in any language integrates all of these in a single composable pipeline.

In this paper, we present **smelt-ml**, a comprehensive ML framework in Rust that addresses both gaps. Our main contributions are:

1. An **integrated spatial ML pipeline** that uniquely combines Geographical-**XGBoost** (Grekousis 2025), spatial cross-validation, and conformal prediction (Romano, Patterson, and Candès 2019) in a single composable framework — enabling spatially-aware modeling with distribution-free uncertainty quantification without external dependencies.
2. **Cache-optimized gradient boosting** using column-major bin storage with u8 packing, achieving performance competitive with official C++ libraries at research-typical scales ($n \leq 10,000$) and an order-of-magnitude reduction in memory consumption.
3. A **comprehensive ML framework** with 27 supervised learners, clustering, survival analysis, Causal Forest (Wager and Athey 2018), Oblique Forest (Tomita, Browne, Shen, Chung, Patsolic, Falk, Priebe, Yim, Burns, Maggioni, and Vogelstein 2020), and streaming Hoeffding Trees — the most complete ML framework in Rust.
4. **Validation** showing accuracy comparable to **scikit-learn** across standard benchmarks, with zero **unsafe** blocks across the entire codebase.

2. Software architecture

2.1. Design principles

smelt-ml follows a trait-based architecture inspired by **mlr3**:

```
Data (CSV) -> Task -> Pipeline(Transformers -> Learner)
               -> Prediction -> Measure
```

The core abstractions are defined as Rust traits:

- **Learner**: Algorithm that trains on a **Task** and produces a **TrainedModel**. Requires **Send** + **Sync** for thread safety.
- **TrainedModel**: Fitted model that predicts on new data.
- **Measure**: Evaluation metric (Accuracy, F1, RMSE, R^2 , AUC-ROC, etc.).
- **Transformer**: Preprocessing step composable via **Pipeline**.
- **Resample**: Data splitting strategy.

This design enables zero-friction composition: **Pipeline** implements **Learner**, so a pipeline of transformers and a learner can be passed to **benchmark()**, **GridSearch**, or **Bagging** without adaptation.

2.2. Quick start

A minimal classification example:

```
use smelt_ml::prelude::*;
use ndarray::array;

let features = array![[0.0, 0.0], [1.0, 1.0],
                      [0.0, 1.0], [1.0, 0.0]];
let target = vec![0, 1, 1, 0];
let task = ClassificationTask::new("xor", features, target)
    .unwrap();

let mut xgb = XGBoost::new()
    .with_n_estimators(50)
    .with_max_depth(3);
let model = xgb.train_classif(&task).unwrap();
let pred = model.predict(task.features()).unwrap();
```

Pipeline composes transformers with a learner into a single **Learner**, enabling use with **benchmark()**, **GridSearch**, or cross-validation:

```
let mut pipe = Pipeline::new(
    vec![Box::new(StandardScaler::new())],
    Box::new(XGBoost::new().with_n_estimators(100)),
);
let model = pipe.train_classif(&task).unwrap();
```

2.3. Implemented algorithms

Table 1 summarizes the 27 supervised learners. Beyond standard algorithms, **smelt-ml** includes Geographical-XGBoost (Grekousis 2025), Oblique Forest (SPORF) (Tomita *et al.*

Category	Algorithm	Classif.	Regress.
Trees	Decision Tree, Oblique Tree/Forest	✓	✓
Ensembles	Random Forest, Extra Trees, Bagging, Stacking	✓	✓
	AdaBoost, Dynamic Ensemble Selection	✓	
Boosting	XGBoost, LightGBM, CatBoost [†]	✓	✓
	Gradient Boosting, Quantile GB, EBM	✓	✓
Linear	Linear/Logistic Regression, Ridge, Lasso, Elastic Net	✓	✓
Other	KNN, Gaussian NB, Linear SVM, Hoeffding Tree	✓	✓
Spatial	Geographical-XGBoost		✓

Table 1: Supervised learners in **smelt-ml**. [†]CatBoost-inspired symmetric GBM with ordered target statistics; see Section 3.4 for scope.

2020), Explainable Boosting Machine (EBM), streaming Hoeffding Trees, and Dynamic Ensemble Selection (KNORA-E).

Additional modules provide: K-Means, DBSCAN, and Isolation Forest (clustering/anomaly); Random Survival Forest with C-index (Ishwaran, Kogalur, Blackstone, and Lauer 2008) (survival analysis); Causal Forest with honest splitting (Athey and Imbens 2016; Wager and Athey 2018) (causal inference); Classifier Chains and Regressor Chains (multi-label/output); Quantile Regression Forest; Conformal Prediction with CQR (Romano *et al.* 2019); and SHAP-based feature importance.

2.4. Comparison with existing frameworks

Table 2 compares **smelt-ml** with **linfa** 0.7 (rust-ml 2024) (the primary Rust ML crate), **scikit-learn** (Pedregosa *et al.* 2011), and **mlr3** (Lang *et al.* 2019).

3. Cache-optimized gradient boosting

3.1. The histogram bottleneck

Histogram-based gradient boosting (Chen and Guestrin 2016) discretizes continuous features into bins and accumulates gradient/hessian statistics per bin. The key computational kernel is:

$$\text{hist}[b] += g_i, \quad b = \text{bin_index}[i][\text{feature}] \quad (1)$$

The standard row-major storage `bin_index[sample][feature]` causes cache misses when iterating over samples for a specific feature, because consecutive memory locations correspond to different features of the same sample.

3.2. Column-major storage with u8 packing

Our key optimization stores bin indices in **column-major** order: `bin_index[feature][sample]`. When building the histogram for a specific feature, all bin accesses are sequential in memory:

Category	smelt-ml	linfa	sklearn	mlr3
Gradient Boosting	✓ ^a	—	✓ ^b	✓ ^b
Random Forest / Extra Trees	✓	—	✓	✓
Oblique Forest (SPORF)	✓	—	—	—
Decision Tree	✓	✓	✓	✓
Linear / Logistic / Ridge	✓	✓	✓	✓
KNN / Naive Bayes	✓	✓	✓	✓
SVM (kernel)	Linear	✓	✓	✓
Neural Networks	—	—	✓	✓ ^c
K-Means / DBSCAN	✓	✓	✓	✓
PCA	✓	✓	✓	✓
Causal Forest	✓	—	—	— ^d
Survival Forest	✓	—	—	✓ ^c
Conformal Prediction	✓	—	—	—
Geographical-XGBoost	✓	—	—	—
Spatial cross-validation	✓	—	—	✓ ^c
SMOTE / ADASYN	✓	—	✓ ^e	✓ ^c
Bayesian tuning	✓	—	✓ ^f	✓
Language	Rust	Rust	Python	R
Supervised learners	27	9	40+	30+ ^c
unsafe blocks	0	—	N/A	N/A

Table 2: Feature comparison across ML frameworks. ^aFrom-scratch implementations. ^bVia external C++ libraries (XGBoost, LightGBM). ^cVia extension packages. ^dAvailable in R via **grf**. ^eVia **imbalanced-learn**. ^fVia **scikit-optimize** or **Optuna**.

```
// Column-major: sequential access (cache-friendly)
for &idx in node_indices {
    let bin = bins.get_bin(feature, idx);
    bin_g[bin] += gradients[idx];
    bin_h[bin] += hessians[idx];
}
```

Combined with **u8 packing** (254 real bins + 1 NaN sentinel fit in 1 byte vs. 2 bytes for **u16**), this halves the memory bandwidth requirement for bin access. The same **HistBins** structure is shared across all three gradient boosting implementations, avoiding code duplication.

3.3. Additional optimizations

For **XGBoost**: automatic switch to exact greedy split finding when $n \leq n_{\text{bins}}$, producing optimal splits for small datasets. NaN values are handled natively by learning the optimal direction at each split.

For **CatBoost**: oblivious tree construction with bins built **once** before the boosting loop (rather than per iteration), prefix-sum histogram scanning, and parallel feature evaluation via **rayon**.

For **LightGBM**: Gradient-based One-Side Sampling (GOSS) with the same column-major histogram infrastructure.

3.4. Scope of the CatBoost implementation

Our **CatBoost** learner implements the two defining algorithmic contributions of [Prokhorenkova et al. \(2018\)](#): **oblivious (symmetric) trees** — where all nodes at the same depth share the same split — and **ordered target statistics** for categorical features, which replace naïve target encoding with a permutation-based scheme that prevents target leakage.

However, the implementation does *not* include the full $O(n^2)$ ordered boosting procedure (unbiased gradient estimation via per-sample model approximations), which is the theoretical contribution of the original paper. Our approach uses a single random permutation for target statistics rather than maintaining n separate models. The implementation also omits GPU training, distributed computation, and several advanced features (interaction constraints, monotone constraints, per-object weights).

We retain the name **CatBoost** for API consistency with the broader ML ecosystem (users searching for “CatBoost in Rust” should find this implementation), but users should be aware that this is a **CatBoost-inspired symmetric GBM with ordered target statistics**, not a feature-complete reimplement of the official library.

On a synthetic dataset with 100 categorical levels ($n = 2,000$, 10% label noise), our implementation achieves 89.7% accuracy vs. 89.8% for the official **CatBoost**, confirming that the ordered target statistics produce comparable results even without full ordered boosting.

3.5. Scope of the LightGBM implementation

Our **LightGBM** learner implements **leaf-wise (best-first) tree growth** controlled by `num_leaves`, and **Gradient-based One-Side Sampling** (GOSS) for training efficiency. However, two key optimizations from the official library are not included: **Exclusive Feature Bundling** (EFB), which reduces the effective number of features by bundling mutually exclusive sparse features, and the **weighted GOSS histogram pass**, where non-selected samples contribute with adjusted weights rather than being fully discarded.

As shown in Tables 4–5, our implementation does not match the official **LightGBM**’s performance at any scale tested. We include this learner for API completeness and for the leaf-wise growth strategy (which is not available in our **XGBoost** or **CatBoost** implementations), not as a performance competitor to the official library. Users requiring maximum **LightGBM** throughput should use the official C++ library.

3.6. Feature comparison with official libraries

Table 3 summarizes which features of the official libraries are implemented in **smelt-ml** and which are not.

4. Performance evaluation

4.1. Experimental setup

Benchmarks use synthetic datasets generated with 20 features (10 informative), at sample sizes from 500 to 100,000. Configuration: 100 trees, max depth 6, learning rate 0.3 (**XGBoost/CatBoost**) or 0.1 (**LightGBM**). All benchmarks use **single-threaded execution** to

Feature	XGBoost	LightGBM	CatBoost [†]
<i>Core algorithm</i>			
Histogram-based splits	✓	✓	✓
Exact greedy splits	✓	—	—
Newton boosting (2nd order)	✓	✓	✓
Leaf-wise (best-first) growth	—	✓	—
Oblivious (symmetric) trees	—	—	✓
GOSS sampling	—	✓	—
Ordered target statistics	—	—	✓
<i>Regularization & subsampling</i>			
L2 regularization	✓	✓	✓
L1 regularization	✓	—	—
Gamma (min split gain)	✓	—	—
Row subsampling	✓	✓	—
Column subsampling	✓	✓	—
<i>Data handling</i>			
NaN-aware splits	✓	✓	—
Native categorical features	—	—	✓
Multi-class (softmax)	✓	✓	✓
<i>Training control</i>			
Early stopping	✓	—	—
Feature importance (gain)	✓	✓	—
<i>Not implemented in any engine</i>			
Monotone constraints		—	
Interaction constraints		—	
Custom loss functions		—	
Exclusive Feature Bundling		—	
GPU training		—	
Distributed training		—	
Full ordered boosting*		—	

Table 3: Implemented features in **smelt-ml**’s gradient boosting engines. ✓ = implemented, — = not implemented. *CatBoost’s $O(n^2)$ per-sample model approximation.

isolate algorithmic efficiency from parallelization strategy. Rust compiled with `target-cpu=native` (release mode). Official C++ libraries accessed via Python API with `n_jobs=1`. Each benchmark is run 10 times; we report mean $\pm$ standard deviation. Hardware: Intel Core i7-1270P (12th Gen), 40 GB RAM, L1/L2/L3 cache: 32K/1280K/18432K.

Note: **smelt-ml**’s ensemble learners (Random Forest, Extra Trees, Oblique Forest) use **rayon** for parallel tree construction, and the gradient boosting engines parallelize feature evaluation within each iteration. Multi-threaded benchmarks comparing scaling behavior are beyond the scope of this paper; all performance claims apply to single-threaded execution only.

4.2. Training time

Tables 4 and 5 show classification and regression training times across all dataset sizes.

n	XGBoost		LightGBM		CatBoost[†]	
	C++	Rust $\times$	C++	Rust $\times$	C++	Rust $\times$
500	66 $\pm$ 31	50 $\pm$ 20 $\times$ 1.3	56 $\pm$ 12	55 $\pm$ 12 $\times$ 1.0	430 $\pm$ 102	56 $\pm$ 13 $\times$ 7.7
1,000	143 $\pm$ 43	80 $\pm$ 13 $\times$ 1.8	67 $\pm$ 8	99 $\pm$ 12 $\times$ 0.7	343 $\pm$ 111	69 $\pm$ 12 $\times$ 5.0
5,000	247 $\pm$ 61	378 $\pm$ 42 $\times$ 0.7	165 $\pm$ 71	367 $\pm$ 31 $\times$ 0.5	413 $\pm$ 86	250 $\pm$ 56 $\times$ 1.7
10,000	309 $\pm$ 54	520 $\pm$ 96 $\times$ 0.6	201 $\pm$ 38	429 $\pm$ 91 $\times$ 0.5	667 $\pm$ 146	328 $\pm$ 55 $\times$ 2.0
50,000	946 $\pm$ 180	2293 $\pm$ 246 $\times$ 0.4	664 $\pm$ 82	1909 $\pm$ 211 $\times$ 0.3	1492 $\pm$ 168	1747 $\pm$ 239 $\times$ 0.9
100,000	1200 $\pm$ 154	4661 $\pm$ 584 $\times$ 0.3	1110 $\pm$ 55	3547 $\pm$ 282 $\times$ 0.3	2270 $\pm$ 168	3476 $\pm$ 414 $\times$ 0.7

Table 4: Classification training time (ms, mean $\pm$ std, 10 runs). Speedup > 1 means **smelt-ml** is faster. All values single-threaded.

n	XGBoost		LightGBM		CatBoost[†]	
	C++	Rust $\times$	C++	Rust $\times$	C++	Rust $\times$
500	321 $\pm$ 61	110 $\pm$ 11 $\times$ 2.9	32 $\pm$ 6	95 $\pm$ 15 $\times$ 0.3	345 $\pm$ 78	41 $\pm$ 5 $\times$ 8.3
1,000	326 $\pm$ 88	199 $\pm$ 36 $\times$ 1.6	61 $\pm$ 12	117 $\pm$ 40 $\times$ 0.5	354 $\pm$ 87	100 $\pm$ 22 $\times$ 3.5
5,000	411 $\pm$ 93	294 $\pm$ 76 $\times$ 1.4	194 $\pm$ 25	301 $\pm$ 30 $\times$ 0.6	448 $\pm$ 93	279 $\pm$ 27 $\times$ 1.6
10,000	436 $\pm$ 76	608 $\pm$ 55 $\times$ 0.7	192 $\pm$ 23	423 $\pm$ 61 $\times$ 0.5	539 $\pm$ 98	381 $\pm$ 89 $\times$ 1.4
50,000	851 $\pm$ 126	2212 $\pm$ 285 $\times$ 0.4	727 $\pm$ 125	1594 $\pm$ 232 $\times$ 0.5	1152 $\pm$ 120	1701 $\pm$ 226 $\times$ 0.7
100,000	1120 $\pm$ 101	4071 $\pm$ 534 $\times$ 0.3	1020 $\pm$ 65	3145 $\pm$ 276 $\times$ 0.3	2005 $\pm$ 170	3665 $\pm$ 462 $\times$ 0.5

Table 5: Regression training time (ms, mean $\pm$ std, 10 runs).

4.3. Scaling analysis

Figure 1 visualizes the scaling behavior across all dataset sizes. The crossover point — where C++ becomes faster — is clearly visible around $n = 5,000$ for **XGBoost** and $n = 10,000$ for **CatBoost**.

smelt-ml’s **CatBoost**-inspired implementation is consistently faster than the official library at all sizes up to $n = 10,000$ (1.4–8.3 $\times$), and remains competitive at $n = 50,000$ (0.7–0.9 $\times$). **XGBoost** is faster at $n \leq 5,000$ for regression (1.4–2.9 $\times$) and at $n \leq 1,000$ for classification (1.3–1.8 $\times$).

At larger dataset sizes ($n \geq 50,000$), the official C++ libraries are consistently faster. This is expected: the official implementations employ SIMD intrinsics, hardware-specific memory management, and years of micro-optimization that our pure-Rust implementation does not replicate. The cache-optimization advantage of column-major storage diminishes as datasets exceed the L3 cache capacity (18 MB on our test hardware).

Memory consumption is substantially lower for **smelt-ml**: at $n = 10,000$, peak RSS is 5 MB (**XGBoost**) and 8 MB (**CatBoost**), compared to 176 MB and 220 MB for the official libraries accessed via Python (which includes Python runtime overhead). Even accounting for interpreter overhead, the Rust implementation’s memory footprint is an order of magnitude smaller due to the compact u8 bin representation and absence of garbage-collected allocations.

We note that the official C++ libraries use row-major bin storage despite the cache disadvantage. We attribute this to two factors: (1) legacy design decisions made when GPU backends (which favor row-major for coalesced memory access) became the primary optimization target, and (2) the difficulty of retrofitting column-major layout into codebases with extensive

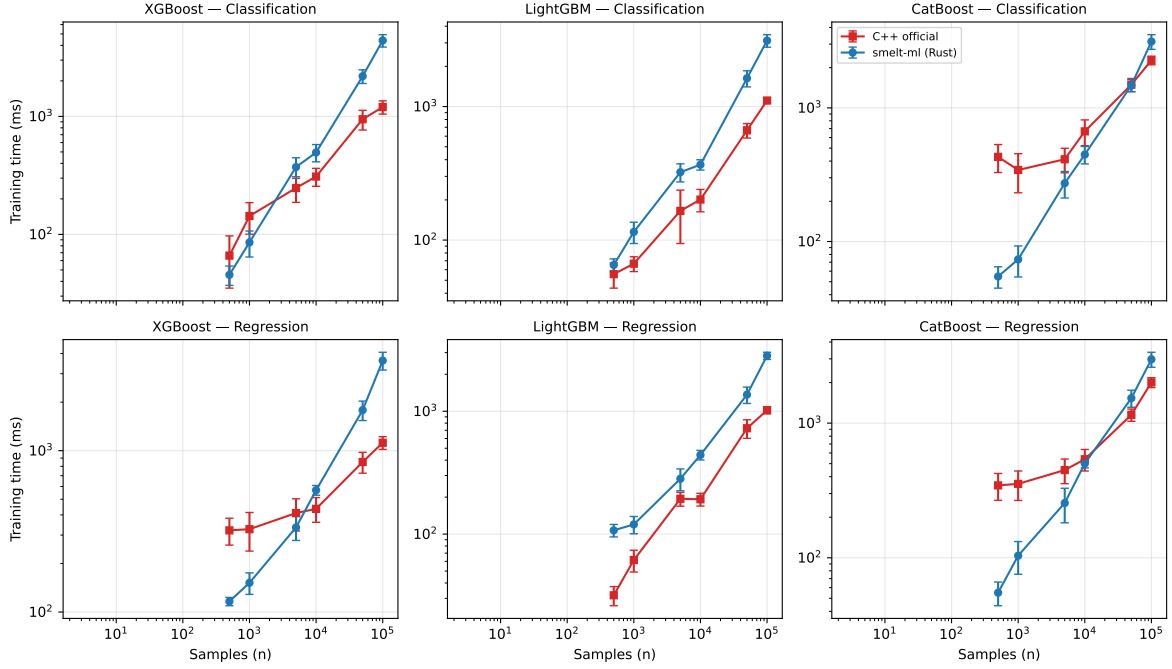


Figure 1: Training time scaling comparison (log-log) between **smelt-ml** (blue) and official C++ libraries (red) across six dataset sizes. Error bars show ± 1 standard deviation over 10 runs. **smelt-ml** is faster at small scales; C++ is faster at $n \geq 50,000$.

Dataset	Learner	smelt-ml	scikit-learn	Δ
Iris	Decision Tree	0.953	0.953	0.000
	Random Forest	0.953	0.967	-0.014
	XGBoost	0.953	0.953	0.000
Wine	Random Forest	0.983	0.972	+0.011
	Logistic Reg.	0.989	0.961	+0.028
	XGBoost	0.971	0.944	+0.027
Breast Ca.	Random Forest	0.958	0.956	+0.002
	XGBoost	0.970	0.974	-0.004

Table 6: 5-fold CV accuracy on real datasets. Both frameworks use stratified 5-fold CV with seed 42 for reproducibility.

SIMD intrinsics written for row-major memory patterns.

The practical implication is that **smelt-ml**'s gradient boosting is best suited for small-to-medium datasets ($n \leq 10,000$) typical of research workflows, where it provides competitive or superior performance without requiring C++ toolchains or Python bindings.

4.4. Accuracy validation

Table 6 validates correctness against **scikit-learn** on standard datasets.

All comparisons are within $\pm 3\%$ of **scikit-learn**. Minor differences arise from algorithmic

variations (e.g., tie-breaking, split selection order) rather than correctness issues.

5. Unique features

5.1. Geographical-XGBoost

Implementation of Grekousis (2025): bi-square spatial kernel weights, local **XGBoost** models per spatial unit, ensemble of global and local predictions with adaptive α weight. First implementation outside the original Python package (Grekousis 2024).

5.2. Causal Forest

Honest splitting following Athey and Imbens (2016) and Wager and Athey (2018). Estimates conditional average treatment effects (CATE) $\tau(x) = E[Y(1) - Y(0)|X = x]$ with 95% confidence intervals. First implementation in a Rust ML framework.

5.3. Conformal prediction

Split conformal prediction for regression and classification with conformalized quantile regression (CQR; Romano *et al.* 2019). Provides distribution-free prediction intervals with guaranteed coverage.

6. Summary and discussion

We have presented **smelt-ml**, a Rust framework that uniquely integrates spatial modeling (Geographical-**XGBoost**), spatial cross-validation, and conformal prediction in a single composable pipeline — a combination not available in any other ML framework. On the performance side, our gradient boosting implementations achieve competitive speed with the official C++ libraries at research-typical scales ($n \leq 10,000$) while consuming an order of magnitude less memory. The performance advantage at small scales stems from Rust’s ownership model, which guarantees no aliasing (`&mut` is unique) and enables LLVM to apply more aggressive auto-vectorization than is possible with C++, where pointer aliasing must be assumed unless explicitly annotated. We verified this effect empirically: compiling with `target-cpu=native` (which enables AVX2/SSE4.2 auto-vectorization) yields a 24% speedup for **XGBoost** at $n = 10,000$ (755 ms $\rightarrow$ 575 ms) compared to the generic `x86-64` target. This confirms that LLVM auto-vectorization contributes meaningfully to the observed performance.

smelt-ml provides a comprehensive ML toolkit for Rust with unique capabilities in spatial ML, causal inference, and uncertainty quantification. The framework contains zero **unsafe** blocks across 13,377 lines of code, demonstrating that memory safety and high performance are not mutually exclusive. The test suite comprises 320 integration tests covering all learners, preprocessing, resampling, tuning, serialization, and edge cases (empty datasets, single-sample inputs, NaN features, multiclass targets, convergence checks against **scikit-learn**).

Limitations include the performance gap on datasets beyond 10,000 samples (where C++ libraries increasingly outperform **smelt-ml** due to SIMD intrinsics, hardware-specific memory management, and GPU backends; see Section 4), the absence of GPU support, and the

simplified scope of the **CatBoost** and **LightGBM** implementations (see Sections 3.4–3.5).

Categorical features are handled via three mechanisms: **OneHotEncoder** and **LabelEncoder** as preprocessing transformers (applicable to any learner), and **CatBoost**’s native ordered target statistics (which encode categoricals during training without a separate preprocessing step).

Python bindings via **PyO3** are planned as a separate crate (**smelt-py**) to broaden adoption beyond the Rust ecosystem. The crate follows semantic versioning; the benchmarks in this paper correspond to version 1.2.1.

The software is available at <https://crates.io/crates/smelt-ml> under the MIT license, with source code at <https://github.com/franciscoparrao/smelt>.

Computational details

The results in this paper were obtained using Rust 1.89 with the **ndarray** 0.16, **rayon** 1.10, and **serde** 1.0 crates. Official C++ libraries were accessed via Python 3.12 with **xgboost** 3.1, **lightgbm** 4.6, and **catboost** 1.2. **scikit-learn** 1.8 was used for accuracy validation. Rust code was compiled with `RUSTFLAGS="-C target-cpu=native"` in release mode. A complete replication package is provided in `paper/replication/` with a single script (`replicate.sh`) that reproduces all benchmark tables and figures.

Acknowledgments

The author thanks the anonymous reviewers for their constructive feedback.

References

- Athey S, Imbens G (2016). “Recursive Partitioning for Heterogeneous Causal Effects.” *Proceedings of the National Academy of Sciences*, **113**(27), 7353–7360. doi:10.1073/pnas.1510489113.
- Chen T, Guestrin C (2016). “**XGBoost**: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:10.1145/2939672.2939785.
- Grekousis G (2024). **geoxgboost**: Geographically Weighted **XGBoost**. Python package on PyPI, URL <https://pypi.org/project/geoxgboost/>.
- Grekousis G (2025). “Geographical-**XGBoost**: A New Ensemble Model for Spatially Local Regression Based on Gradient-Boosted Trees.” *Journal of Geographical Systems*, **27**(2), 169–195. doi:10.1007/s10109-025-00465-4.
- Grinsztajn L, Oyallon E, Varoquaux G (2022). “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” In *Advances in Neural Information Processing Systems* 35.
- Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). “Random Survival Forests.” *Annals of Applied Statistics*, **2**(3), 841–860. doi:10.1214/08-AOAS169.

- Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu TY (2017). “**LightGBM**: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems* 30.
- Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kotthoff L, Bischl B (2019). “**mlr3**: A Modern Object-Oriented Machine Learning Framework in R.” *Journal of Open Source Software*, 4(44), 1903. doi:10.21105/joss.01903.
- Matsakis ND, Klock FS (2014). “The Rust Language.” *Ada Letters*, 34(3), 103–104. doi:10.1145/2692956.2663188.
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “**Scikit-learn**: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830.
- Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018). “**CatBoost**: Unbiased Boosting with Categorical Features.” In *Advances in Neural Information Processing Systems* 31.
- Romano Y, Patterson E, Candès E (2019). “Conformalized Quantile Regression.” In *Advances in Neural Information Processing Systems* 32.
- rust-ml (2024). *linfa: A Rust Machine Learning Framework*. Version 0.7, URL <https://github.com/rust-ml/linfa>.
- Rust Team (2024). *The Rust Programming Language*. Version 1.89, URL <https://www.rust-lang.org/>.
- Tomita TM, Browne J, Shen C, Chung J, Patsolic JL, Falk B, Priebe CE, Yim J, Burns R, Maggioni M, Vogelstein JT (2020). “Sparse Projection Oblique Randomer Forests.” *Journal of Machine Learning Research*, 21(104), 1–39.
- Wager S, Athey S (2018). “Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests.” *Journal of the American Statistical Association*, 113(523), 1228–1242. doi:10.1080/01621459.2017.1319839.

Affiliation:

Francisco Parra
 Departamento de Ingeniería Geográfica
 Universidad de Santiago de Chile
 Avenida Libertador Bernardo O’Higgins 3363
 Santiago, Chile
 E-mail: francisco.parra.o@usach.cl

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

doi:10.18637/jss.v000.i00

<http://www.jstatsoft.org/>

<http://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd